

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Data Science and Business Informatics

Project 29

Multi-class SVM with a Lagrangian Dual and Bundle Methods

Part I: formulation

Professor:

Prof. Antonio Frangioni

Students:

Tolgonai Nasipbek kyzy

Hamse Hassan Adnan

Group 26 — Project 29

ANNO ACCADEMICO 2025/2026

Assignment, as originally stated

Project 29

(M) is a Support Vector Machine-type approach of your choice (in particular, with one or more kernels of your choice) for multi-class classification, e.g., using the standard One-to-Rest approach.

(A1) is a dual approach, cf. constrained minimization slides, applied to either the primal or the dual formulation of the SVM, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Specific discussion should be provided about if and how solving one traning problem in the One-to-Rest approach may provide useful solution that may help in solving the others. Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, and of course for (A2).

Group 26

Tolgonai Nasipbek kyzy, 626449, t.nasipbekkyzy@studenti.unipi.it
Hamse Hassan Adnan, 683187, h.adnan@studenti.unipi.it

Contents

1 Introduction **3**
- 1.1 Description of the task . . . 3

2 SVM Problem Formulation **4**
- 2.1 Soft margin formulation . . . 4
- 2.2 Dual formulation . . . 4
- 2.3 Primal and dual variables . . . 5
- 2.4 Kernel trick . . . 6

3 The Lagrangian Dual **7**
- 3.1 Which constraints to dualise . . . 7
- 3.2 Dualising the representer identity . . . 7
- 3.3 The relaxed problem is a continuous knapsack . . . 8
- 3.4 Properties of the dual function . . . 8
- 3.5 What this dual actually is . . . 9
- 3.6 Relation to the direct nonsmooth formulation . . . 9

4 The Bundle Method **10**
- 4.1 The cutting-plane model and the bundle . . . 10
- 4.2 The structured master problem . . . 10
- 4.3 The variant we use . . . 11
- 4.4 Primal recovery and Dantzig–Wolfe . . . 12
- 4.5 Kernelised form . . . 12
- 4.6 Cost, and how many cuts are needed . . . 12

5 The General-Purpose Solver **13**

6 Multiclass SVM **14**
- 6.1 One-vs-Rest approach . . . 14
- 6.2 Does solving one problem help the others? . . . 14

7 What to expect from the algorithms **15**
- 7.1 Assumptions, and whether we satisfy them . . . 15
- 7.2 Convergence and complexity . . . 15
- 7.3 A preliminary check . . . 16

8 What we are asking **17**

A The shared-slack-budget model **18**

1. Introduction

1.1 Description of the task

This report explores two different optimization algorithms applied to a machine learning model.

The ML model (M) is a Support Vector Machine-type approach for multi-class classification, with a choice of kernels, using the standard One-vs-Rest approach.

The first optimization algorithm (A1) is a dual approach applied to the dual formulation of the SVM. We reintroduce the weight vector w as an explicit variable, so that the representer identity $w = \sum_i \alpha_i y_i \phi(x_i)$ becomes a constraint of the problem, and we dualise *that* identity. The resulting Lagrangian relaxation is a linear program over a box with a single equality constraint — a continuous knapsack, solved exactly by sorting in $O(m \log m)$ — so the dual function is finite everywhere, genuinely non-differentiable, and defined on a space of dimension n for a linear kernel or m in general. It is maximised by a proximal bundle method, whose master problem is solved by an off-the-shelf QP solver, as the project statement allows.

The second optimization algorithm (A2) is a general-purpose convex QP solver applied to the standard SVM dual. No off-the-shelf solver is used elsewhere, save for the master problem of the bundle method.

Specific discussion is provided in Chapter 6 about if and how solving one training problem in the One-vs-Rest approach may provide useful solutions that help in solving the others.

2. SVM Problem Formulation

The goal of this chapter is to describe the Support Vector Machine and to derive the Lagrangian dual formulation that defines the optimization task as a quadratic programming problem. The material is standard; it is included because Chapter 3 departs from it at one specific point, the stationarity condition (2.2), and that departure is easier to follow with the usual derivation in front of the reader.

2.1 Soft margin formulation

Let $\{(x_i, y_i)\}_{i=1}^m$ with $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, +1\}$, and let $\phi : \mathbb{R}^n \to \mathcal{H}$ be a feature map into a Hilbert space with reproducing kernel $\kappa(x, x') = \langle \phi(x),\, \phi(x') \rangle$. Write $\phi_i := \phi(x_i)$.

A separating hyperplane $H(w, b) = \{x : \langle w,\, \phi(x) \rangle + b = 0\}$ classifies correctly with margin at least one when $y_i(\langle w,\, \phi_i \rangle + b) \geq 1$ for all i, and the geometric margin is then $1/\|w\|$, so maximising the margin means minimising $\|w\|$. Since such a hyperplane need not exist, each sample is given a slack $\xi_i \geq 0$ by which it may fall short of its own margin, penalised linearly with weight $C > 0$. This gives the soft-margin (C-)SVM:

$$\begin{aligned} \min_{w \in \mathcal{H},\, b \in \mathbb{R},\, \xi \in \mathbb{R}^m} \quad & \tfrac{1}{2}\|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y_i\big(\langle w,\, \phi_i \rangle + b\big) \;\geq\; 1 - \xi_i, \qquad i = 1, \ldots, m, \\ & \xi_i \;\geq\; 0, \qquad i = 1, \ldots, m. \end{aligned} \tag{P}$$

Here $\|w\|$ measures model complexity and C weighs the empirical loss against generalisation. Problem (P) is a convex QP, strictly convex in w but only convex in (b, ξ).

Eliminating ξ gives the equivalent unconstrained, non-smooth form

$$\min_{w,b} \; \tfrac{1}{2}\|w\|^2 + C \sum_{i=1}^m \big[1 - y_i(\langle w,\, \phi_i \rangle + b)\big]^+, \qquad [t]^+ := \max\{0, t\}, \tag{2.1}$$

which is the form in which the SVM appears in the course material as the motivating example for nondifferentiable optimization. We shall meet it again, unexpectedly, in Section 3.5.

2.2 Dual formulation

Attach $\alpha_i \geq 0$ to the margin constraints and $\mu_i \geq 0$ to $\xi_i \geq 0$:

$$L(w, b, \xi; \alpha, \mu) = \tfrac{1}{2}\|w\|^2 + C\mathbf{1}^\top \xi + \sum_{i=1}^m \alpha_i\big(1 - \xi_i - y_i(\langle w,\, \phi_i \rangle + b)\big) - \mu^\top \xi.$$

Stationarity in (w, b, ξ) gives the three conditions

$$w = \sum_{i=1}^m \alpha_i y_i \phi_i, \qquad \sum_{i=1}^m \alpha_i y_i = 0, \qquad \alpha_i + \mu_i = C. \tag{2.2}$$

The first is the *representer identity*; it is the hinge on which this whole project turns — Chapter 3 dualises precisely this condition.

Substituting the three conditions back into L makes every term collapse. Writing $K_{ij} = \kappa(x_i, x_j)$, $Y = \operatorname{diag}(y)$ and $Q = YKY$:

$$\tfrac{1}{2}\|w\|^2 = \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} = \tfrac{1}{2}\alpha^\top Q \alpha,$$

$$-\sum_i \alpha_i y_i \langle w,\, \phi_i \rangle = -\Big\langle w,\, \sum_i \alpha_i y_i \phi_i \Big\rangle = -\|w\|^2 = -\,\alpha^\top Q \alpha,$$

$$-\,b \sum_i \alpha_i y_i = 0 \quad \text{(by the } b \text{ condition)},$$

$$\sum_i \xi_i \big(C - \alpha_i - \mu_i\big) = 0 \quad \text{(by the } \xi \text{ condition)},$$

leaving $\mathbf{1}^\top \alpha - \tfrac{1}{2}\alpha^\top Q \alpha$, to be maximised. Two remarks on how the constraints of the dual arise, because both are easy to state loosely:

- $\mu_i \geq 0$ and $\mu_i = C - \alpha_i$ together give $\alpha_i \leq C$, which with $\alpha_i \geq 0$ is the box $0 \leq \alpha \leq C\mathbf{1}$;
- b enters L only through the term $-b\sum_i \alpha_i y_i$, and b is *free*, so $\inf_b L = -\infty$ unless $\sum_i \alpha_i y_i = 0$. The b-stationarity condition therefore does not disappear when it is used to cancel that term: it survives as the constraint $y^\top \alpha = 0$.

Writing the result as a minimisation of the negative, the dual of (P) is the convex QP

$$\min_{\alpha \in \mathbb{R}^m} \quad \tfrac{1}{2}\,\alpha^\top Q\,\alpha - \mathbf{1}^\top \alpha \qquad \text{s.t.} \qquad \alpha \in D := \big\{\alpha :\ y^\top \alpha = 0,\ \ 0 \leq \alpha \leq C\mathbf{1}\big\}. \tag{D}$$

D is a non-empty ($0 \in D$) compact polytope and $Q \succeq 0$, so (D) always has a solution. Slater's condition holds for (P), hence there is no duality gap and the KKT conditions

$$\begin{aligned} \alpha_i = 0 \quad &\Longrightarrow \quad y_i f(x_i) \geq 1, \\ 0 < \alpha_i < C \quad &\Longrightarrow \quad y_i f(x_i) = 1, \\ \alpha_i = C \quad &\Longrightarrow \quad y_i f(x_i) \leq 1, \end{aligned} \tag{2.3}$$

characterise optimality, where $f(x) = \sum_i \alpha_i y_i \kappa(x_i, x) + b$. Samples with $\alpha_i > 0$ are the *support vectors*; those with $0 < \alpha_i < C$ are the *free* support vectors. Their number will turn out to control the cost of (A1); see Section 4.6.

2.3 Primal and dual variables

Given an optimal $\alpha^\star$ of (D), the primal variables come back as

$$w^\star = \sum_i \alpha_i^\star y_i \phi_i, \qquad \xi_i^\star = \big[\, 1 - y_i(\langle w^\star,\, \phi_i \rangle + b^\star) \,\big]^+,$$

the first directly from the representer identity and the second from primal feasibility of the margin constraint together with complementarity on $\xi_i \geq 0$ (note the factor y_i inside: ξ_i measures the violation of *the sample's own* margin, not of the hyperplane equation).

The bias $b^\star$ deserves more care. The textbook recipe reads $b^\star$ off any free support vector via (2.3) and averages over them for stability; but the average is only as good as the set of indices identified as free, so a solver that is merely close to optimal produces a b that degrades the measured primal–dual gap. The better observation is that b enters the Lagrangian only through $-b\sum_i \alpha_i y_i$, so b *is the multiplier of the equality constraint*

$y^\top \alpha = 0$. Any algorithm that produces that multiplier explicitly can use it directly, with no support-vector bookkeeping at all. For an interior-point method it is one of the iterates; in our case the knapsack of Section 3.3 supplies it at every oracle call, as (3.3) proves by LP duality.

2.4 Kernel trick

Only inner products appear in (D), so ϕ need never be formed. The kernels used in this report are

- linear: $\kappa(x, x') = x^\top x'$;
- polynomial: $\kappa(x, x') = (\gamma\, x^\top x' + c_0)^d$, $\gamma > 0$;
- Gaussian (RBF): $\kappa(x, x') = \exp(-\gamma \|x - x'\|^2)$, $\gamma > 0$.

Unlike the linear case, for a nonlinear kernel $\mathcal{H}$ may be infinite-dimensional; Section 4.5 explains how the dual variable of (A1) is represented in that case.

3. The Lagrangian Dual

3.1 Which constraints to dualise

For a Lagrangian dual approach to be worth building, the relaxation must satisfy:

(R1) the relaxed problem must be *substantially easier* than the original — ideally closed-form or combinatorial;

(R2) the dual must be *multi-dimensional*, otherwise a one-dimensional search solves it and bundle machinery is pointless;

(R3) the dual should be *genuinely non-differentiable*, otherwise a smooth method is preferable.

Relaxing the single equality $y^\top \alpha = 0$ in (D) violates (R1) and (R2): what is left is

$$\min\Big\{ \tfrac{1}{2}\alpha^\top Q\alpha - \mathbf{1}^\top\alpha + \theta\, y^\top\alpha \;:\; 0 \le \alpha \le C\mathbf{1} \Big\},$$

a box-constrained QP with the *same dense Hessian* — the unbiased SVM, no easier than the original — and the dual is univariate. Relaxing the box $\alpha \le C\mathbf{1}$ fares no better: the relaxation is again a QP in Q.

The reason is structural: in (D) *no constraint is the difficulty*. The difficulty is Q itself, a dense $m \times m$ matrix. So the object to dualise is not a constraint of (D) at all — it is the identity that *created* Q.

3.2 Dualising the representer identity

Q appears in (D) only because w was eliminated through the representer identity (2.2). Put w back:

$$\min_{\alpha,\, w} \quad \tfrac{1}{2}\|w\|^2 - \mathbf{1}^\top\alpha \qquad \text{s.t.} \qquad w = \sum_{i=1}^{m} \alpha_i y_i \phi_i, \qquad \alpha \in D. \tag{D$'$}$$

Substituting the constraint recovers (D) exactly, so (D$'$) and (D) are the same problem. Now dualise *only* the linking identity, with a free multiplier $\lambda \in \mathcal{H}$:

$$L(\alpha, w; \lambda) = \tfrac{1}{2}\|w\|^2 - \mathbf{1}^\top\alpha + \Big\langle \lambda,\; w - \sum_i \alpha_i y_i \phi_i \Big\rangle.$$

The two groups of variables separate. Minimising over $w \in \mathcal{H}$ is unconstrained and gives $w(\lambda) = -\lambda$ with value $-\tfrac{1}{2}\|\lambda\|^2$; what remains is linear in α over D. Hence

$$\boxed{\psi(\lambda) \;=\; -\tfrac{1}{2}\|\lambda\|^2 \;+\; \min_{\alpha \in D}\; c(\lambda)^\top \alpha, \qquad c_i(\lambda) = -\big(1 + y_i \langle \lambda,\, \phi_i \rangle\big),} \tag{3.1}$$

and the Lagrangian dual of (D$'$) is $\max_{\lambda \in \mathcal{H}} \psi(\lambda)$.

In the vocabulary of partial Lagrangian relaxation, the representer identity is the *complicating* constraint — it is what couples w to α and so what creates the dense Q — while $\alpha \in D$ are the *easy* constraints, kept in the relaxed problem because they are exactly what makes it a knapsack. The usual warning that $\psi(\lambda)$ may be $-\infty$ for some λ, and that one then has to neutralise the offending directions of zero curvature, does not apply here: D is compact, so the relaxed problem always has a finite optimum and ψ is finite on all of $\mathcal{H}$.

3.3 The relaxed problem is a continuous knapsack

The inner problem in (3.1) is

$$\min\ c^\top \alpha \qquad \text{s.t.} \qquad y^\top \alpha = 0, \quad 0 \le \alpha \le C\mathbf{1}, \tag{KP}$$

a linear program over a box with a *single* equality constraint: the classical *continuous knapsack* problem. Relaxing that one equality with a scalar θ makes the box minimiser explicit,

$$\alpha_i(\theta) = \begin{cases} C, & c_i - \theta y_i < 0, \\ 0, & c_i - \theta y_i > 0, \\ \text{any value in } [0, C], & c_i - \theta y_i = 0, \end{cases}$$

and $h(\theta) := y^\top \alpha(\theta)$ is non-decreasing, so a $\theta^\star$ with $h(\theta^\star) = 0$ exists and is found by sorting the m breakpoints $y_i c_i$ and scanning (or by median search in $O(m)$ expected time). Splitting the tied components to make the equality hold exactly produces an α that minimises the Lagrangian of (KP) *and* is feasible, hence optimal. Forming c costs one kernel product; the rest costs $O(m \log m)$. This settles (R1): a dense QP has been replaced by a sort.

The same structure — relax a single equality, obtain a separable problem whose solution is piecewise linear in the multiplier with one breakpoint per variable — appears as a worked exercise in the course material for the projection onto a hyperplane, and (KP) is its bounded-variable analogue.

3.4 Properties of the dual function

Proposition 3.1 (finiteness)**.** *D is a non-empty compact polytope, so the minimum in* (3.1) *is attained and finite for every λ. Hence $\psi : \mathcal{H} \to \mathbb{R}$ is finite everywhere, concave and upper semicontinuous.*

This is unusually convenient: no feasibility ("infinite") cuts are ever needed, and every oracle call returns a valid lower bound on the optimal value.

Proposition 3.2 (structure and non-differentiability)**.** *Write*

$$\psi = \psi_{\mathrm{sm}} + \psi_{\mathrm{pl}}, \qquad \psi_{\mathrm{sm}}(\lambda) = -\tfrac{1}{2}\|\lambda\|^2, \quad \psi_{\mathrm{pl}}(\lambda) = \min_{\alpha \in D} c(\lambda)^\top \alpha.$$

Then ψ_{sm} is smooth and known in closed form, and ψ_{pl} is a minimum of finitely many affine functions of λ (one per vertex of D), hence concave polyhedral. ψ fails to be differentiable exactly where (KP) *has multiple optima, i.e. where some reduced cost vanishes, $c_i - \theta^\star y_i = 0$.*

Unwinding that condition with $w = -\lambda$ gives $y_i \langle w, \phi_i \rangle + \theta^\star y_i = 1$: the kinks of ψ sit *exactly* on the support-vector condition $y_i f(x_i) = 1$ of (2.3). Since any non-degenerate SVM has free support vectors, the maximiser of ψ *is* a point of non-differentiability. This settles (R3): the non-smoothness is not incidental, it is the solution.

Proposition 3.3 (no duality gap)**.** (D′) *is convex, its coupling constraint is affine and D is compact; therefore* $\max_\lambda \psi(\lambda) = v(\mathrm{D}') = v(\mathrm{D})$*, and the maximiser is $\lambda^\star = -w^\star$.*

The multiplier of the relaxed identity is thus, up to sign, the primal weight vector itself. A supergradient of ψ at λ is the violation of the relaxed constraint,

$$g(\lambda) = w(\lambda) - \sum_i \alpha_i(\lambda) y_i \phi_i = -\lambda - \sum_i \alpha_i(\lambda) y_i \phi_i, \tag{3.2}$$

available at no extra cost from the oracle. (R2) holds as well: λ ranges over $\mathcal{H}$, of dimension n for the linear kernel and m after the reduction of Section 4.5 in general.

3.5 What this dual actually is

It is worth identifying $\max_\lambda \psi$ in closed form. Substituting $w = -\lambda$ in (3.1),

$$\max_{\lambda} \psi(\lambda) \iff -\min_{w} \Big\{ \tfrac{1}{2}\|w\|^2 + \max_{\alpha \in D} \sum_i \alpha_i \big(1 - y_i \langle w,\, \phi_i \rangle\big) \Big\},$$

and dualising the single equality inside the inner maximisation with a scalar θ gives, by LP duality,

$$\max_{\alpha \in D} \sum_i \alpha_i \big(1 - y_i g_i\big) = \min_{\theta}\; C \sum_i \big[1 - y_i (g_i + \theta)\big]^+, \qquad g_i := \langle w,\, \phi_i \rangle. \tag{3.3}$$

So the Lagrangian dual of (D′) is the hinge-loss primal (2.1), and *the multiplier θ of the knapsack equality is the SVM bias b* — so b falls out of the oracle for free. That the dual of the dual is the primal is of course biconjugacy; the next section says what is nevertheless gained.

3.6 Relation to the direct nonsmooth formulation

The SVM appears in the course material on nondifferentiable unconstrained optimization as the motivating example, in the form

$$\min_{w,b} \quad \|w\|^2 + C \sum_{i \in I} \max\big\{1 - y_i(\langle w,\, x_i \rangle - b),\, 0\big\},$$

whose nondifferentiability is precisely what motivates bundle methods. Since Section 3.5 shows that $\max_\lambda \psi$ is that same function, we must say clearly what dualising the representer identity adds over simply running a bundle method on it — which is also what the published cutting-plane methods for regularised risk minimisation do (Joachims 2006; Teo et al. 2010; Franc & Sonnenburg 2008).

The answer is *not* that the bias makes a direct approach impossible: $f(w, b)$ is convex in (w, b) jointly and a subgradient in both variables is immediate, so one can perfectly well put b in the bundle. The answer is that our relaxation minimises over b *exactly*, at every oracle call, instead of approximating the b direction with cutting planes. The knapsack multiplier θ of (3.3) *is* that exact partial minimisation, and the sort of Section 3.3 is how it is computed. Equivalently: we bundle $g(w) = \min_b f(w, b)$ rather than $f(w, b)$, and g is the better-conditioned object because one of its directions has been eliminated in closed form. The measured effect is reported in Chapter 7.

We claim exactly this much: the family of cuts and the function being optimised are the same as in the direct approach, and what relaxing the representer identity adds is (i) the exact elimination of b at every oracle call, (ii) the SVM dual variables through the aggregated solution of Section 4.4, and (iii) a two-sided bound on $v(\mathrm{D})$ rather than a bound on one's own objective only.

4. The Bundle Method

The goal of this chapter is to describe the algorithm we use to maximise ψ, and to show what the master problem gives us besides the optimal multiplier. We first recall the cutting-plane model and its stabilised master, then specialise both to our ψ, and finally read off the primal solution.

4.1 The cutting-plane model and the bundle

We recall the method only far enough to fix notation, following the course material [1] and [2]; our setting is concave maximisation, so every inequality below is reversed with respect to the usual convex statement.

The oracle at λ returns $\psi(\lambda)$ and a supergradient $g(\lambda)$. Since ψ is concave, one call bounds it *everywhere*, $\psi(\mu) \le \ell_\lambda(\mu) := \psi(\lambda) + \langle g(\lambda), \mu - \lambda \rangle$ for all μ, and this global validity is what makes first-order information useful here. The collected triples form the *bundle* $\mathcal{B} = \{(\lambda^h, \psi^h, g^h)\}$, whose pointwise minimum $\psi_\mathcal{B} \ge \psi$ is the *cutting-plane model*; maximising it is the *master problem*. Because the bare model over-estimates where few cuts sit, the master is stabilised around a *stability centre* $\bar{\lambda}$ by a proximal term with parameter t, and a *serious step* moves the centre when the true ψ delivers a fixed fraction κ of the predicted improvement, a *null step* keeping it and the new cut otherwise. The course material writes the stabilisation with $\mu = 1/t$ multiplying rather than dividing.

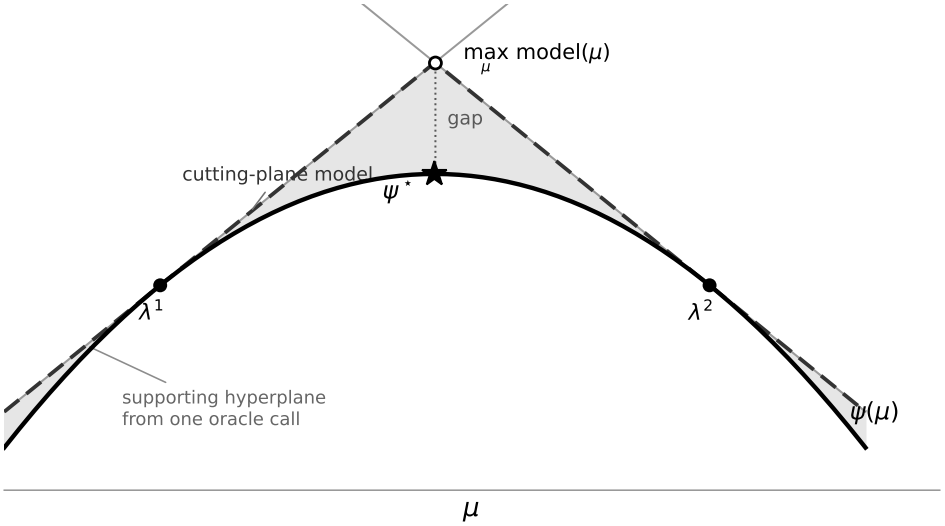


Figure 4.1: The horizontal axis is μ, the free variable of the inequality above. Here λ^1 and λ^2 are the two points at which the oracle has been called, and the drawing is the one-dimensional case, so that the supporting hyperplanes appear as lines. One oracle call at λ returns $\psi(\lambda)$ and a supergradient, and hence a hyperplane that lies above ψ *everywhere*, not merely near λ: this is what concavity buys. The model is the pointwise minimum of the hyperplanes collected so far, so it too lies above ψ, and its maximum is an upper bound on $\psi^\star$. Each new call adds a hyperplane, which can only lower the minimum. The method probes next where the model is highest, since that is the only place the maximum can be.

4.2 The structured master problem

We depart from the generic scheme of Section 4.1 in one respect. Writing $\psi = \psi_{\mathrm{sm}} + \psi_{\mathrm{pl}}$ as in Chapter 3, the smooth part $\psi_{\mathrm{sm}}(\lambda) = -\frac{1}{2}\|\lambda\|^2$ is known in closed form. Approximating

it with cuts gains nothing and loses something, because a piecewise-linear model of a curved function is poor near its maximiser. We therefore linearise only ψ_{pl} and carry ψ_{sm} into the master unchanged. This is a *generalized*, or structured, bundle method. Each oracle call returns some $\alpha^j \in D$, which gives the affine upper model

$$\psi_{\mathrm{pl}}(\lambda) \;\le\; c(\lambda)^\top \alpha^j = -\,\mathbf{1}^\top \alpha^j - \langle \lambda,\, \textstyle\sum_i \alpha_i^j y_i \phi_i \rangle =: -a_j - \langle \lambda,\, b_j \rangle.$$

With a stability centre $\bar\lambda$ and proximal parameter $t > 0$ the master is

$$\max_{\lambda \in \mathcal{H}} \;\; -\tfrac{1}{2}\|\lambda\|^2 + \min_{j=1,\dots,J} \bigl\{ -a_j - \langle \lambda,\, b_j \rangle \bigr\} - \tfrac{1}{2t}\|\lambda - \bar\lambda\|^2. \tag{4.1}$$

Exchanging min and max and eliminating λ gives its dual, a QP over the unit simplex Δ^J:

$$\min_{\vartheta \in \Delta^J} \;\; \frac{1}{2\rho} \Bigl\| \tfrac{\bar\lambda}{t} - \textstyle\sum_j \vartheta_j b_j \Bigr\|^2 - \;\textstyle\sum_j \vartheta_j a_j, \qquad \rho := 1 + \tfrac{1}{t}, \tag{4.2}$$

after which $\lambda^{\mathrm{new}} = \bigl(\bar\lambda/t - \sum_j \vartheta_j b_j\bigr)/\rho$. Only the $J \times J$ Gram matrix $G_{jl} = \langle b_j,\, b_l \rangle$ and the vector $p_j = \langle b_j,\, \bar\lambda \rangle$ are needed, and we maintain both incrementally, so a master costs $O(J^2)$ plus one kernel product per new cut. We solve it with an off-the-shelf QP solver, which the project statement allows.

Our variant differs from the generic scheme in two respects. The smooth part is carried in the master exactly rather than cut, as above, and we stop on the *certified gap* of Section 4.4, which brackets $v(\mathrm{D})$ from both sides at no extra cost, rather than on the usual test on the length of the step.

4.3 The variant we use

Three choices complete the description of the algorithm.

Initialisation. We start from $\lambda = 0$, which is admissible because ψ is finite everywhere. The first oracle call then minimises $-\mathbf{1}^\top \alpha$ over D, so the first cut is generated by a vertex of D at which as much mass as the equality permits sits at the upper bound, and $\psi(0)$ is already a valid lower bound on $v(\mathrm{D})$.

Keeping the bundle bounded. Left alone the bundle grows by one cut per iteration, and the master grows with it. We cap the number of cuts. When the cap is reached the oldest cuts are replaced by the single *aggregate* cut generated by $\hat\alpha = \sum_j \vartheta_j \alpha^j$, which is legitimate because $\hat\alpha \in D$, so the aggregate is as valid a cut as any the oracle returns. Since Section 4.6 argues that the number of cuts needed tracks the number of free support vectors, the cap has to be set above that number, and we will report what happens when it is not.

The serious-step test. Writing $\psi_{\mathcal{B}}$ for the model of Section 4.2, that is the exact quadratic plus the cuts, and λ^{new} for the master's solution, we take a serious step when

$$\psi(\lambda^{\mathrm{new}}) - \psi(\bar\lambda) \;\ge\; \kappa \bigl[\psi_{\mathcal{B}}(\lambda^{\mathrm{new}}) - \psi(\bar\lambda) \bigr], \qquad \kappa \in (0,1), \tag{4.3}$$

and a null step otherwise. Since $\psi_{\mathcal{B}} \ge \psi$ everywhere, the bracket is the improvement the model predicted and is non-negative, so the test asks that the true function deliver at least the fraction κ of what was promised.

Parameters. The variant has four: the proximal parameter t, the fraction κ of (4.3), the cap on the bundle, and the tolerance on the certified gap, the run stopping when the bracket of Section 4.4 falls below it. We increase t after a serious step and decrease it after a null step, which is the usual heuristic. Their effect is one of the things the experiments have to establish, since none of them has a value that theory fixes.

4.4 Primal recovery and Dantzig–Wolfe

Let $\hat{\alpha} := \sum_j \vartheta_j \alpha^j$ be the *aggregated solution* built from the master problem's dual multipliers. Since $\vartheta \in \Delta^J$ and D is convex, $\hat{\alpha} \in D$: it is a *feasible* solution of (D) at every iteration. Moreover $\sum_j \vartheta_j b_j = \Phi^\top Y \hat{\alpha}$ and $\sum_j \vartheta_j a_j = \mathbf{1}^\top \hat{\alpha}$, so for $t \to \infty$ (4.2) becomes

$$\min_{\vartheta \in \Delta^J} \ \tfrac{1}{2}\, \hat{\alpha}^\top Q \hat{\alpha} - \mathbf{1}^\top \hat{\alpha},$$

i.e. *the SVM dual* (D) *restricted to* $\operatorname{conv}\{\alpha^1, \ldots, \alpha^J\} \subseteq D$. The method is therefore exactly *Dantzig–Wolfe decomposition* [5] of (D): the scheme that optimises over a polytope by keeping an inner approximation of it, here $\operatorname{conv}\{\alpha^j\}$, and enlarging that approximation with points supplied by a cheap subproblem, called the *pricing problem*, which here is (KP). Generating those points one at a time is *column generation*. Bundle on ψ, cutting planes on (2.1) and column generation on (D) are thus three views of one algorithm. Two practical consequences:

- the master returns the primal solution $\hat{\alpha} \to \alpha^\star$, so we need nothing extra to recover the classifier,
- $\psi(\lambda) \le v(\mathrm{D}) \le \tfrac{1}{2}\hat{\alpha}^\top Q \hat{\alpha} - \mathbf{1}^\top \hat{\alpha}$ at *every* iteration, a **certified gap**, which decomposition algorithms for the SVM such as SMO do not provide.

4.5 Kernelised form

For a general kernel $\mathcal{H}$ may be infinite-dimensional, so write $\lambda = \sum_j \beta_j \phi_j$. This is without loss of generality: a component of λ orthogonal to $\operatorname{span}\{\phi_i\}$ leaves $c(\lambda)$ unchanged and only increases $\|\lambda\|^2$, so at the optimum it is zero. Then $\langle \lambda, \phi_i \rangle = (K\beta)_i$ and $\|\lambda\|^2 = \beta^\top K \beta$, and one oracle call costs one matrix–vector product with K plus a sort. Note that the correct stabilising term is the *RKHS* norm $\|\lambda - \bar{\lambda}\|^2 = (\beta - \bar{\beta})^\top K (\beta - \bar{\beta})$ rather than the Euclidean $\|\beta - \bar{\beta}\|^2$. It is the K-metric that makes (4.2) come out in the form above.

4.6 Cost, and how many cuts are needed

Per iteration: one oracle call ($O(m^2)$ for the kernel product, $O(m \log m)$ for the sort) plus one master ($O(J^2)$ in the QP, $O(Jm)$ to update G). The interesting question is how large J must grow. The oracle returns essentially *vertices* of D, that is $0/C$ vectors, and $\hat{\alpha}$ is a convex combination of them: a component of $\alpha^\star$ sitting at a bound is produced by a single vertex, but every *free* support vector $0 < \alpha_i^\star < C$ must be produced by mixing vertices that disagree on i. We therefore expect the number of cuts the method needs to be governed by the number of free support vectors, and truncating the bundle below that number to make the gap stall, which is the classical Dantzig–Wolfe tailing-off. If that is right, the bundle has to be sized above the expected number of free support vectors, which large C and simple kernels keep small, and the count of free support vectors becomes the quantity that predicts the cost of a run. Measuring this is part of the experimental work to come.

5. The General-Purpose Solver

For (A2) the dual (D) is handed as a dense convex QP to an off-the-shelf interior-point solver. Mapping it onto the generic QP interface $\min\{\frac{1}{2}z^\top Hz + f^\top z : A_{\mathrm{eq}}z = b_{\mathrm{eq}},\ l \le z \le u\}$ requires only

$$H = Q, \qquad f = -\mathbf{1}, \qquad A_{\mathrm{eq}} = y^\top, \qquad b_{\mathrm{eq}} = 0, \qquad l = 0, \qquad u = C\mathbf{1},$$

and the bias is read off as the multiplier of the single equality row, exactly as in Section 2.3. This is our reference for solution quality and for running time.

Both algorithms have been implemented, and the experimental comparison between them is the subject of the second part of this work.

6. Multiclass SVM

6.1 One-vs-Rest approach

The SVM is a binary classifier. With q classes and labels $c_i \in \{1, \dots, q\}$, One-vs-Rest solves q independent binary problems: for class k set

$$y_i^k = \begin{cases} +1, & c_i = k \\ -1, & c_i \neq k \end{cases}$$

and solve (P)/(D) with $y = y^k$, obtaining $f^k(x) = \sum_i \alpha_i^k y_i^k \kappa(x_i, x) + b^k$. A new point is assigned to $\arg\max_k f^k(x)$. It constructs q models, against $q(q-1)/2$ for One-vs-One. See [8] for a comparison of the two.

Note that three observations matter for what follows:

1. The q problems share the same kernel matrix K. They differ only through the label vector, since $Q^k = Y^k K Y^k$ and D^k depends on k only through the single equality $(y^k)^\top \alpha = 0$. Q^k is never formed.

2. The classifiers are trained independently, but at prediction time we compare them on a common scale that nothing in the training enforces. This produces the familiar One-vs-Rest pathologies: ties ($f^k(x) > 0$ for two k) and rejections ($f^k(x) < 0$ for all k).

3. Each binary problem is unbalanced, roughly $1 : (q-1)$.

6.2 Does solving one problem help the others?

The project statement asks this explicitly. The q problems differ only through y^k, so there are three candidate mechanisms. They are not equally useful, and we take them in turn.

The kernel matrix. K is computed once and shared. We never form $Q^k = Y^k K Y^k$, because the oracle only ever needs $K\beta$. Better still, the q oracles can be run in lockstep so that their q matrix–vector products become one BLAS-3 product $K\,[\beta^1 \cdots \beta^q]$. This is a genuine, structural saving of a factor q in the dominant cost.

Warm-starting the multiplier. Since $\lambda^\star = -w^\star$, warm-starting class $k{+}1$ at class k's optimum amounts to assuming the two separating hyperplanes are similar. They are not: the positive class changes completely between the two problems.

Transferring cuts. Any point of D^l generates a valid cut for class l, and the D^k differ only through the equality. Writing P_k, P_l for the two positive blocks and R for the rest, $\alpha \in D^k$ satisfies $\alpha(P_k) = \alpha(P_l) + \alpha(R)$, so $(y^l)^\top \alpha = -2\,\alpha(R)$: the imbalance is known in closed form and can be repaired in $O(m)$ by moving mass onto P_l, *preserving the support pattern* of the transferred cut. This costs no oracle call and no kernel product.

Only the first of the three is certain to pay. Sharing the kernel matrix saves a factor q in the dominant cost, and that saving is structural rather than circumstantial. The other two are plausible but not obviously useful, and we are cautious about both. Warm-starting the multiplier assumes that consecutive hyperplanes are similar, which they need not be, since the positive class changes completely from one problem to the next, and a stability centre inherited from the wrong place costs iterations to undo. Transferring cuts does no harm, but a repaired cut need not be an informative one. Which of the three is worth its complexity is a question for the experiments.

7. What to expect from the algorithms

This chapter states the convergence and complexity properties we expect to see, the assumptions they rest on, and whether our problem satisfies them.

7.1 Assumptions, and whether we satisfy them

The convergence theory of proximal bundle methods [2, 3] assumes a convex (here concave) objective that is finite on the whole space, an oracle returning its value and one subgradient, and bounded subgradients on the region explored. We check these in turn.

Concavity holds: ψ is a minimum of affine functions of λ plus the concave term $-\frac{1}{2}\|\lambda\|^2$ (Section 3.4).

Finiteness holds, and this is the assumption that most often fails in Lagrangian relaxation. Because the relaxed problem is a minimisation over the compact polytope D, its optimum is attained for every λ, so $\psi > -\infty$ everywhere. We therefore never need the feasibility cuts, nor the neutralisation of zero-curvature directions, that the course material warns about for relaxations whose inner problem can be unbounded.

Differentiability does *not* hold, and is not assumed. It fails exactly where the knapsack has ties, which is the support-vector condition $y_i f(x_i) = 1$, and the maximiser is such a point. This is why a bundle method is used rather than a gradient method.

Bounded subgradients. The supergradient is $-\lambda - \Phi^\top Y \alpha(\lambda)$ with $\alpha(\lambda) \in D$ and $\|\alpha\|_\infty \leq C$, so on any bounded set of multipliers the supergradients are bounded, and ψ is Lipschitz there.

No duality gap. Problem (D′) is convex, its coupling constraint is affine and D is compact, so $\max_\lambda \psi = v(\mathrm{D})$. What the bundle method converges to is therefore the value we want, not a bound on it.

7.2 Convergence and complexity

With these assumptions the proximal bundle method converges to the optimum, and the complexity that the course material quotes for the nonsmooth convex case is $O(1/\varepsilon^2)$ iterations, the same order as the subgradient method, with the practical difference lying in the constant and in the behaviour in the tail. We expect the tail behaviour to matter here: the bundle accumulates cuts around the maximiser, and the smooth part $-\frac{1}{2}\|\lambda\|^2$ is carried in the master exactly rather than approximated.

The strong concavity of that term is worth isolating, because it is what the published analyses of cutting-plane methods for regularised risk minimisation [10, 9, 11] exploit to obtain $O(1/\varepsilon)$ rather than $O(1/\varepsilon^2)$. Since Section 3.5 shows that our dual is that same regularised risk, we expect their rate to apply to (A1) as well.

Per-iteration cost. One oracle call is one kernel product, $O(m^2)$ in general and $O(mn)$ for a linear kernel, plus an $O(m \log m)$ sort. One master problem is a quadratic program over the unit simplex in J variables, costing $O(J^2)$ in the solver and $O(Jm)$ to update the Gram matrix incrementally. We expect J to grow with the number of free support vectors, for the reason given in Section 4.6, so the dominant term should be $O(m^2)$ per iteration on kernel problems.

For (A2) the interior-point solver on a dense convex quadratic program of order m costs $O(m^3)$ per iteration in the factorisation, in a number of iterations that is in practice nearly independent of m. This is the trade-off the experiments are meant to expose: (A1) pays little per iteration and may need many, (A2) pays a great deal per iteration and needs few.

7.3 A preliminary check

We have implemented both algorithms, and although the experimental study belongs to the later parts of the report, two things are worth reporting here because Section 3.4 of the project information asks whether we expect these properties to matter in practice.

The first concerns the assumptions above. Strong duality holds in our runs to a relative error below $4 \cdot 10^{-9}$ for the three kernels and $C \in \{0.1, 1, 10\}$, the knapsack agrees with a general LP solver to $1.4 \cdot 10^{-12}$, and the aggregated solution and the knapsack multiplier reproduce the $\alpha^\star$ and b of a general-purpose QP solver to about 10^{-7}. The formulation is therefore not merely plausible on paper.

The second concerns the claim of Section 3.6, that eliminating b exactly should cost fewer iterations than approximating that direction with cuts. Comparing (A1) with a bundle method applied directly to $f(w, b)$, with b as a bundle variable and everything else held equal, gives:

dataset	kernel	b in the bundle	b eliminated (ours)
WINE	linear	137	29
WINE	Gaussian	400 (gap $6 \cdot 10^{-5}$)	133 (gap $1.1 \cdot 10^{-8}$)
IRIS	Gaussian	96	24
CANCER	linear	220	52

Table 7.1: Iterations to a relative gap of 10^{-8} on a first, small set of instances. The Gaussian WINE baseline had not reached tolerance at the 400-iteration cap.

A factor of four to five, and one instance where the direct method does not reach tolerance at all. We report this as an indication rather than a result: four instances is not a study, and the proper comparison, over a realistic range of sizes and with the parameter sensitivity, belongs to the experimental part.

A caveat on the citations. We quote these results from the sources listed, and before the final version we will verify each against the original and give the theorem and page number, as the project guidelines require.

8. What we are asking

Section 3.1 onwards has set out the proposal; we close by stating the one point on which a decision would help us most, and what follows from it.

The point is the one raised in Section 3.6. The maximum of our dual is the hinge-loss primal, which is where the course material begins when it motivates nondifferentiable optimization. It is fair to object that we have dualised our way back to the problem we started from and are running a known cutting-plane method on it. Our answer is that the dualisation buys three things: the bias is eliminated exactly at every oracle call rather than approximated by cuts, the master problem's multipliers return a feasible $\hat{\alpha} \in D$ and so the SVM dual variables themselves, and ψ together with the restricted problem bracket $v(\mathrm{D})$ from both sides at every iteration. We claim no more than that. If it is judged too thin to count as a different dual approach, we would rather know now than after the experimental work, which is why we are sending this part on its own.

If the formulation is accepted, the remaining parts of the report will cover the experimental set-up and the results: the comparison between (A1) and (A2), how the bundle grows with the number of free support vectors, which of the three One-vs-Rest transfer mechanisms is worth its complexity, and timings and parameter sensitivity.

A. The shared-slack-budget model

For completeness we record an alternative model that couples the One-vs-Rest problems, together with a defect we found in it. Give each sample a single slack budget B_i to be shared by all q binary classifiers:

$$\begin{aligned} \min_{w,b,\xi} \quad & \sum_{k=1}^{q} \tfrac{1}{2}\|w^k\|^2 + C\sum_{i,k}\xi_i^k \\ \text{s.t.} \quad & y_i^k\big(\langle w^k,\,\phi_i\rangle + b^k\big) \;\geq\; 1-\xi_i^k, \qquad \xi_i^k \;\geq\; 0, \qquad \sum_{k=1}^{q}\xi_i^k \;\leq\; B_i. \end{aligned} \tag{M-B}$$

Dualising only the mq margin constraints with $\alpha_i^k \geq 0$ separates the variables. Minimising over w^k gives the usual $w^k = \sum_i \alpha_i^k y_i^k \phi_i$ and $-\frac{1}{2}(\alpha^k)^\top Q^k \alpha^k$, minimising over b^k forces $\sum_i \alpha_i^k y_i^k = 0$, and minimising over ξ splits across i into a tiny LP that is solved by inspection, putting the whole budget on the most negative coefficient or using none of it. Hence

$$\psi(\alpha) = \sum_{i,k}\alpha_i^k - \tfrac{1}{2}\sum_k (\alpha^k)^\top Q^k \alpha^k - \sum_i B_i\Big[\max_k\big(\alpha_i^k - C\big)\Big]^+, \tag{A.1}$$

on $\{\alpha \geq 0 :\ \sum_i \alpha_i^k y_i^k = 0 \ \ \forall k\}$. The budget replaces the box $\alpha \leq C$ by a penalty, and the dual is non-differentiable where two or more classes tie for a contested budget, which is where the One-vs-Rest coupling happens.

Writing this out showed us a defect. With a hard budget (M-B) can be infeasible, and its Lagrangian dual is then unbounded. A sample lying where two classifiers both score near zero needs $\xi_i^k \approx 1$ from each, so $B_i < 2$ is usually infeasible already and one outlier destroys the training problem. The repair is to soften the budget, $\sum_k \xi_i^k \leq B_i + z_i$ with $z_i \geq 0$ penalised by Dz_i: the ξ subproblem stays an LP, the model is always feasible, and boundedness in z restores the box $0 \leq \alpha \leq (C+D)\mathbf{1}$.

We do not propose it as the main line. Even repaired it keeps the quadratic forms Q^k inside the dual function, so the bundle has to approximate a curved function with planes, whereas the relaxation of Chapter 3 leaves an LP. And it changes (M), so any comparison with One-vs-Rest would confound a modelling change with an algorithmic one.

Bibliography

[1] A. Frangioni. *Optimization for Data Science* (784AA), course material, Università di Pisa. (Decks on nondifferentiable unconstrained optimization and on constrained optimization.)

[2] A. Frangioni. Standard Bundle Methods: Untrusted Models and Duality. In *Numerical Nonsmooth Optimization: State of the Art Algorithms*, A.M. Bagirov, M. Gaudioso, N. Karmitsa, M. Mäkelä, S. Taheri (Eds.), 61–116, Springer, 2020.

[3] J.-B. Hiriart-Urruty, C. Lemaréchal. *Convex Analysis and Minimization Algorithms I–II*. Springer-Verlag, 1993.

[4] J.E. Kelley. The cutting-plane method for solving convex programs. *Journal of the SIAM* 8(4):703–712, 1960.

[5] G.B. Dantzig, P. Wolfe. Decomposition principle for linear programs. *Operations Research* 8(1):101–111, 1960.

[6] S. Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2, 2015.

[7] C. Cortes, V. Vapnik. Support-vector networks. *Machine Learning* 20(3):273–297, 1995.

[8] C.-W. Hsu, C.-J. Lin. A comparison of methods for multi-class support vector machines. *IEEE Transactions on Neural Networks* 13(2):415–425, 2002.

[9] T. Joachims. Training linear SVMs in linear time. In *Proceedings of KDD*, 2006.

[10] C.H. Teo, S.V.N. Vishwanathan, A. Smola, Q.V. Le. Bundle methods for regularized risk minimization. *Journal of Machine Learning Research* 11:311–365, 2010.

[11] V. Franc, S. Sonnenburg. Optimized cutting plane algorithm for support vector machines. In *Proceedings of ICML*, 2008.